## Практическая работа №3. Форматы данных

Цель работы: на практике изучить текстовые и бинарные форматы сериализации данных (JSON, XML, YAML, MessagePack, CBOR, Protobuf), сравнить их размеры, скорость и читаемость; освоить передачу данных по сети (TCP) и понять принципы схемной эволюции.

### 1. Ответы на вопросы

**Чем отличаются текстовые и бинарные форматы?**

Текстовые форматы (JSON, XML, YAML) записывают данные в виде символов, которые может прочитать человек, поэтому их удобно отлаживать. Но они избыточны: имена полей, кавычки, скобки и теги повторяются, а числа записываются цифрами, из-за чего данные занимают больше места и разбираются медленнее. Бинарные форматы (Protobuf, MessagePack, CBOR) кодируют те же данные байтами в компактном виде, поэтому они меньше и быстрее обрабатываются, но без специальных инструментов их нельзя прочитать.

**Что такое схемная эволюция?**

Схемная эволюция – это способность формата менять структуру данных (добавлять и удалять поля) так, чтобы старые и новые программы продолжали работать вместе и не ломались. Например, новый код читает старые данные без нового поля, а старый код игнорирует незнакомое ему поле.

### 2. Результаты выполнения шагов

Шаг 1. Подготовка проекта. В папке «date_format» созданы все необходимые файлы: исходные данные «data.py», схема «profile.proto», сгенерированный командой «protoc --python_out=. profile.proto» файл «profile_pb2.py», скрипты тестов «step_*.py», «server.py», «client.py» и «run_all.py».

Шаг 2. Текстовые форматы. Профиль пользователя сериализован в JSON, XML и YAML; во всех случаях десериализация прошла успешно.

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python step_json.py
=== JSON ===
{
  "id": 42,
  "name": "Анна Смирнова",
  "age": 22,
  "is_student": true,
  "courses": [
    "Математика",
    "Физика",
    "Информатика"
  ],
  "address": {
    "city": "Москва",
    "zip": "101000"
  }
}
Размер: 253 байт
Десериализация успешна: Анна Смирнова
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python step_xml.py
=== XML ===
<profile><id>42</id><name>Анна Смирнова</name><age>22</age><is_student>True</is_
student><courses><item>Математика</item><item>Физика</item><item>Информатика</it
em></courses><address><city>Москва</city><zip>101000</zip></address></profile>
Размер: 283 байт
Имя пользователя: Анна Смирнова
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python step_yaml.py
=== YAML ===
id: 42
name: Анна Смирнова
age: 22
is_student: true
courses:
- Математика
- Физика
- Информатика
address:
  city: Москва
  zip: '101000'

Размер: 182 байт
Десериализация успешна: Анна Смирнова
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

Шаг 3. Бинарные форматы. Те же данные сериализованы в MessagePack, CBOR и Protobuf (по схеме «profile.proto»); выведено шестнадцатеричное представление и размер.

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python step_msgpack.py
=== MessagePack ===
HEX: 86a269642aa46e616d65b9d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0a36
1676516aa69735f73747564656e74c3a7636f757273657393b4d09cd0b0d182d0b5d0bcd0b0d182d
0b8d0bad0b0acd0a4d0b8d0b7d0b8d0bad0b0b6d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad
0b0a76164647265737382a463697479acd09cd0bed181d0bad0b2d0b0a37a6970a6313031303030
Размер: 157 байт
Десериализация успешна: Анна Смирнова
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python step_cbor.py
=== CBOR ===
HEX: a6626964182a646e616d657819d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b
063616765166a69735f73747564656e74f567636f75727365738374d09cd0b0d182d0b5d0bcd0b0d
182d0b8d0bad0b06cd0a4d0b8d0b7d0b8d0bad0b076d098d0bdd184d0bed180d0bcd0b0d182d0b8d
0bad0b06761646472657373a264636974796cd09cd0bed181d0bad0b2d0b0637a697066313031303
030
Размер: 159 байт
Десериализация успешна: Анна Смирнова
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python step_protobuf.py
=== Protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd
0b0d182d0b5d0bcd0b0d182d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d
0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd09cd0bed181d0bad0b2d0b01206313031303030
Размер: 117 байт
Десериализация успешна: Анна Смирнова
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

Шаг 4. Передача данных по сети (TCP). Сервер запущен в первом терминале, клиент – во втором. Клиент отправляет формат (1 байт), длину (4 байта) и сами данные. Сервер выводит HEX, TEXT и DATA и сохраняет сообщение в папку «received/». У JSON поле TEXT читаемо. У Protobuf в TEXT видны только обрывки строк и служебные символы, а структура данных не читается (поле DATA получено только после разбора по схеме). У MessagePack байты вообще не декодируются как текст, поэтому сервер выводит «<не текст, бинарные данные>».

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python server.py
Сервер слушает 127.0.0.1:9999...
Подключился: ('127.0.0.1', 50412)

=== Получено 211 байт, формат: json ===
HEX: 7b226964223a2034322c20226e616d65223a2022d090d0bdd0bdd0b020d0a1d0bcd0b8d180d
0bdd0bed0b2d0b0222c2022616765223a2032322c202269735f73747564656e74223a20747275652
c2022636f7572736573223a205b22d09cd0b0d182d0b5d0bcd0b0d182d0b8d0bad0b0222c2022d0a
4d0b8d0b7d0b8d0bad0b0222c2022d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b0225d2c2
022616464726573732223a207b2263697479223a2022d09cd0bed181d0bad0b2d0b0222c20227a697
0223a20223130313030227d7d
TEXT: {"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "course
s": ["Математика", "Физика", "Информатика"], "address": {"city": "Москва", "zip"
: "101000"}}
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'course
s': ['Математика', 'Физика', 'Информатика'], 'address': {'city': 'Москва', 'zip'
: '101000'}}
[СОХРАНЕНО] received/message_json.json (211 байт)
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python client.py
Отправка в формате json, размер: 211 байт
Отправлено.
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python server.py
Сервер слушает 127.0.0.1:9999...
Подключился: ('127.0.0.1', 50419)

=== Получено 117 байт, формат: protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd
0b0d182d0b5d0bcd0b0d182d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d
0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd09cd0bed181d0bad0b2d0b01206313031303030
TEXT: *Анна Смирнова *Математика*Физика*Информатика2
Москва101000
DATA: {'id': 42, 'name': 'Анна Смирнова', 'courses': ['Математика', 'Физика', 'И
нформатика']}
[СОХРАНЕНО] received/message_protobuf.pb (117 байт)
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python client.py
Отправка в формате protobuf, размер: 117 байт
Отправлено.
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python server.py
Сервер слушает 127.0.0.1:9999...
Подключился: ('127.0.0.1', 50431)

=== Получено 157 байт, формат: msgpack ===
HEX: 86a269642aa46e616d65b9d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0a36
1676516aa69735f73747564656e74c3a7636f757273657393b4d09cd0b0d182d0b5d0bcd0b0d182d
0b8d0bad0b0acd0a4d0b8d0b7d0b8d0bad0b0b6d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad
0b0a7616464726573738 2a463697479acd09cd0bed181d0bad0b2d0b0a37a6970a6313031303030
TEXT: <не текст, бинарные данные>
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'course
s': ['Математика', 'Физика', 'Информатика'], 'address': {'city': 'Москва', 'zip'
: '101000'}}
[СОХРАНЕНО] received/message_msgpack.msgpack (157 байт)
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python client.py
Отправка в формате msgpack, размер: 157 байт
Отправлено.
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

В папке «received/» сохранены файлы разных форматов. Файл «.json» открывается как обычный текст, а «.msgpack» и «.pb» – нет: при выводе их содержимого командой «type» видны лишь отдельные обрывки строк и служебные символы.

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> dir received

    Каталог: C:\backup\Завьялов\ИСПО\Практическая 3\received


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        03.11.2026      3:28            159 message_cbor.cbor
-a----        03.11.2026      3:28            211 message_json.json
-a----        03.11.2026      3:28            157 message_msgpack.msgpack
-a----        03.11.2026      3:28            117 message_protobuf.pb
-a----        03.11.2026      3:34            360 report.json
PS C:\backup\Завьялов\ИСПО\Практическая 3> type received\message_json.json
{"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "courses": ["
Математика", "Физика", "Информатика"], "address": {"city": "Москва", "zip": "101
000"}}
PS C:\backup\Завьялов\ИСПО\Практическая 3> type received\message_msgpack.msgpack
◆◆id*◆name◆Анна Смирнова◆age◆is_studentçcourses◆◆Математика◆Физика◆Информатика◆a
ddress◆◆city◆Москва◆zip◆101000
PS C:\backup\Завьялов\ИСПО\Практическая 3> type received\message_protobuf.pb
*Анна Смирнова *Математика*Физика*Информатика2
Москва101000
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

Шаг 5. Проверка схемной эволюции. В «profile.proto» в конец сообщения «Profile» добавлено поле «string email = 7;», схема

перекомпилирована. Старый файл «message_protobuf.pb», сохранённый до изменения схемы, успешно прочитан новым кодом без ошибок; поле «email» оказалось пустой строкой. Это и есть обратная совместимость.

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> protoc --python_out=. profile.proto
PS C:\backup\Завьялов\ИСПО\Практическая 3> python check_evolution.py
Имя: Анна Смирнова
Email (новое поле): ''
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

Шаг 6. Сводный отчёт. Скрипт «run_all.py» измерил размеры данных во всех форматах.

```
PS C:\backup\Завьялов\ИСПО\Практическая 3> python run_all.py

Формат          Размер (байт)    Читаем?
-----------------------------------------
JSON                      211         да
XML                       283         да
YAML                      182         да
MessagePack               157        нет
CBOR                      159        нет
Protobuf                  117        нет
PS C:\backup\Завьялов\ИСПО\Практическая 3>
```

Таблица 1 – Сравнение текстовых форматов

| **Формат** | **Размер (байт)** | **Читаемость (1-5)** | **Есть ли комментарии?** |
|---|---|---|---|
| JSON | 253 | 5 | Нет |
| XML | 283 | 3 | Да |
| YAML | 182 | 5 | Да |

Размеры взяты из вывода «step_json.py», «step_xml.py» и «step_yaml.py». JSON дан с отступами (indent=2); без отступов его размер – 211 байт.

Таблица 2 – Сводная таблица всех форматов (по выводу run_all.py)

| Формат | Размер (байт) | Читаем ли человеком? | Нужна ли схема? |
|---|---|---|---|
| JSON | 211 | Да | Нет |
| XML | 283 | Да | Нет |
| YAML | 182 | Да | Нет |
| MessagePack | 157 | Нет | Нет |
| CBOR | 159 | Нет | Нет |
| Protobuf | 117 | Нет | Да |

## 3. Выводы

### 1. Почему Protobuf и MessagePack оказались меньше JSON?

В JSON каждое значение сопровождается кавычками, двоеточиями, запятыми, скобками и пробелами, а числа и логические значения записываются текстом. MessagePack заменяет эти символы короткими служебными байтами с типом и длиной, а числа хранит в двоичном виде, поэтому получилось 157 байт против 211. Protobuf экономнее всех (117 байт, примерно на 45% меньше JSON): в данных нет имён полей, вместо них записаны номера полей (1-6) из схемы, а структура известна обеим сторонам заранее. Заметную часть размера везде занимает кириллица: в UTF-8 каждая русская буква занимает 2 байта, и этого ни один формат не уменьшает.

### 2. В каком формате данные было легче всего читать вручную?

Легче всего читаются JSON (с отступами) и YAML: структура видна сразу, лишних символов мало. YAML выглядит чище всего и даже оказался компактнее JSON (182 против 211 байт), так как не использует кавычки и скобки. XML читается хуже, потому что каждое поле обёрнуто в открывающий и закрывающий теги (283 байта). Бинарные форматы вручную не читаются: видны только HEX-значения.

### 3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?

У JSON нет формальной схемы и договорённости о том, как обрабатывать новые и отсутствующие поля: поля опознаются только по

именам внутри данных, типы и значения по умолчанию нигде не заданы. Программа сама должна проверять наличие каждого ключа, иначе при чтении старых данных она упадёт с ошибкой. В Protobuf каждое поле имеет постоянный номер, закреплённый в схеме. Парсер пропускает неизвестные ему номера (старый код читает новые данные), а отсутствующим полям подставляет значения по умолчанию (новый код читает старые данные: «email» стал пустой строкой, как показал шаг 5). Главное правило – не менять и не переиспользовать номера полей.